CMSC 420: Hash Functions and Tables

Justin Wyss-Gallifent

January 20, 2026

1 Problems **2**

1 Problems

1. Suppose $K = \mathcal{Z}$ and T has $m = 7$ entries (hence indexed by $\mathcal{Z}_7$). Define the hash function $h : K \to \mathcal{Z}_7$ by $h(x) = ax \mod m$ for $a = 2$. Suppose we are using closed addressing with linked lists and we set $\lambda_{min} = 0.2$ and $\lambda_{max} = 0.8$. We insert the following integers into the hash table:

$$4, 101, 10, 18, 17, 13, 2, 37$$

This should require exactly one rebuild. When the rebuild happens set the new m accordingly and set a to be the smallest integer greater than 1 which is coprime to the new m.

   (a) Show the hash table (and its linked lists) after the last insert which does not violate the load factor maximum. Explain why there is no violation.

   (b) Show the hash table (and its linked lists) after the last insert which does violate the load factor maximum. Explain why there is a violation.

   (c) Show the hash table after the final insert. Explain why there is no violation.

2. Suppose $p > 3$ is a prime and we define our hash function $h : \mathcal{Z} \to \mathcal{Z}_p$. The explicit definition of h is not relevant here. To resolve collisions we use cubic probing, meaning if $T[h(x)]$ is full we check $T[h(x)+1^3]$, $T[h(x)+2^3]$, $T[h(x)+3^3]$, and so on with additions taken mod p. We claim that cubic probing will probe at least $\left\lfloor \sqrt[3]{p/3} \right\rfloor + 1$ indices.

   (a) Just to make sure you understand what this means, how many indices are guaranteed to be probed for each of $p = 11, 29, 59, 1009$?

   (b) Prove the claim.

3. Suppose $h : K \to \mathcal{Z}_m$ is a hash function which has the property that each index in $\mathcal{Z}_m$ is equally likely to be hit. Assume we are using closed addressing. Suppose n items have been hashed.

   (a) How many empty indices are expected in T? Justify.

   (b) Suppose $m = 100$. Use your answer to (a) to determine the largest n value such that you would expect at least half the indices to be empty.

4. Suppose for load-balancing our hash table we use $\lambda_{min} = 0$ and $\lambda_{max} = 0.5$. We start our hash table with length 1. Suppose we do a sequence of n inserts and each takes time 3. Prove via the aggregate method of amortized analysis that insertion is average case $\mathcal{O}(1)$. Assume that an entire rebuild takes time equal to length of the new table.

5. The following is a really basic (and pretty terrible) simple version of the SHA-256 hash function. It only works on a key x which is a binary string whose length is a multiple of 4.

   (a) Break x up into N sub-keys of length 4 bits: x_1, x_2, ..., x_N.

   (b) Initialize $H = \texttt{0101}$ and $J = \texttt{0111}$.

   (c) For each sub-key x_i do the following:

      i. Update H by XORing it with x_i.

      ii. Update H by XORing it with J.

      iii. Left-rotate J by 1 bit.

   (d) The output is the resulting H, so $h(x) = H$.

   (a) Calculate $h(\texttt{11110100})$. Show the value of H after each step.

   (b) Calculate $h(\texttt{00000000})$. Show the value of H after each step.

   (c) Find the smallest 8-bit binary string which can be prepended to `10101010` so that the output starts with at exactly two `0`s. Do this any way you wish but justify your approach.

6. Suppose you decide to store your data in a hash table with only three entries using closed addressing using standard binary search trees. The keys are integers and the hash function you choose is:

$$h : \mathcal{Z} \to \mathcal{Z}_3 \text{ given by } h(x) = x + 1 \mod 3$$

You insert the following keys in the order given: $1, 5, 3, 7, 10, 9, 4, 2$

Show what the resulting structure looks like. You can illustrate this however you like as long as it is clear.

7. Suppose a hash table has $m = 150$, $\lambda_{min} = 0$, and $\lambda_{max} = 0.73$. We reach the minimum n necessary to rebuild the table. What will the new value of m be? Simplify your answer.

8. Suppose for load-balancing our hash table we use $\lambda_{min} = 0$ and $\lambda_{max} = 1/\alpha$ where $\alpha \geq 2$ is an unknown integer. We start our hash table with length $m = 1$. Suppose we do a sequence of n insertions where $n \geq 1$ is fixed.

   (a) For an unknown number of keys n' if we rebuild, what will the corresponding m' be? Simplify.

   (b) Denote by n_i the ith smallest value for which a rebuild occurs. Calculate n_1, n_2, n_3, n_4 and the corresponding m_i for each. Show work and be organized.

   (c) For an arbitrary $i \geq 1$ what would n_i be and what would the corresponding m_i be? You don't need to prove these - they should be fairly clear from your answers above.

   (d) Suppose insertion costs 1 and rebuilding cost equals the new length allocated. Write down the expression for the total cost required for all n insertions. Simplify but do not evaluate the sum. If your answer has a floor or ceiling, explain why it's there.

   (e) Evaluate the sum.

9. Here is the pseudocode for the hash function from the suggested problems with a `print` statement added. You can assume the length of `x` is a multiple of 4.

```
function hash(x):
    H = 0101
    J = 0111
    break x into n 4-bit chunks x[1],...,x[n]
    for each i = 1 to n inclusive:
        H = H xor x[i]
        H = H xor J
        print H
        J = left rotate J by 1 bit
    end for
    return H
end function
```

(a) Suppose we call `hash(10110101)`. The print statement will execute twice. What will be printed?

First Time	
Second Time	

(b) Find a 4-bit string `x` so `hash(x)` prints `1000`. Explain your process.

10. Suppose a hash table has $m = 200$, $\lambda_{min} = 0$, and $\lambda_{max} = 0.5$. We reach the minimum n necessary to rebuild the table to a new size m. What will n and the new m be? Simplify your answers.

n	
New m	

11. Suppose you decide to store your data in a hash table with only three entries using closed addressing using standard binary search trees. The keys are integers and you are using:

$$h : \mathcal{Z} \to \mathcal{Z}_3 \text{ given by } h(x) = 2x + 1 \mod 3$$

You insert the following keys in the order given: $1, 5, 3, 7, 10, 9, 4, 2$

Show what the resulting structure looks like. You can illustrate this however you like as long as it is clear.